ТОО ”Колледж Хекслет”

Лабораторная работа №5

Работа с файловой системой — асинхронная

Выполнено студентом:

Дубинин Кирилл

Практическое задание

Раздел RO4_1. Работа с файловой системой — асинхронная.

Условие: RO4_1/lesson5/Задание.md

Задания

Общие требования:

• Использовать модуль fs.promises (или require('fs/promises')).

• Весь код должен быть обернут в асинхронные функции (async function).

• Обязательное использование try...catch для обработки ошибок.

• Использовать path.join(__dirname, ...) для путей

Задание 1: Создание структуры. Напишите асинхронную функцию setup(), которая проверяет наличие папки storage. Если её нет — создает её. После создания запишите в файл storage/status.txt строку "Система готова".

Задание 2: Асинхронное чтение. Создайте файл tasks.txt вручную. Напишите функцию, которая асинхронно читает этот файл. Если файл прочитан успешно — выведите его содержимое в верхнем регистре. Если файла нет — выведите "Файл задач не найден", не роняя программу.

Задание 3: Логирование с меткой времени. Реализуйте функцию addLog(message), которая асинхронно дописывает (append) в файл storage/activity.log строку с текущей датой и переданным сообщением.

Задание 4: Цепочка операций. Напишите функцию, которая:

• Читает файл source.txt.

• Создает копию этого файла с названием copy.txt.

• После успешного копирования удаляет оригинал (source.txt).

Задание 5: Параллельная проверка (Promise.all). Дополнительное задание: Создайте массив имен файлов. Используя Promise.all и метод fs.access (или fs.stat), проверьте существование всех этих файлов одновременно и выведите результат.

Ход выполнения

setup ожидает создание каталога и status.txt. readUpper обрабатывает отсутствие файла. addLog дописывает дату и сообщение. makeCopy сначала записывает копию, затем удаляет оригинал; при ошибке записи оригинал

сохраняется. checkFiles ожидает access внутри try и проверяет файлы параллельно через Promise.all.

Проект: RO4_1/lesson5

Из каталога урока: node index.js. Для демонстрации копирования создайте source.txt; остальные операции работают и при его отсутствии.

index.js

index.js

```
const fs = require('fs').promises;
const path = require('path');

// Задание 1: все пути относятся к каталогу лабораторной.
async function setup() {
    try {
        await fs.mkdir(path.join(__dirname, 'storage'), { recursive: true });
        await fs.writeFile(path.join(__dirname, 'storage', 'status.txt'), 'Система
готова', 'utf8');
    } catch (error) { throw new Error(`Подготовка хранилища: ${error.message}`); }
}

// Задание 2: отсутствие tasks.txt не завершает программу аварийно.
async function readUpper() {
    try {
        const content = await fs.readFile(path.join(__dirname, 'tasks.txt'), 'utf8');
        console.log(content.toUpperCase());
        return content.toUpperCase();
    } catch (error) {
        if (error.code === 'ENOENT') { console.log('Файл задач не найден'); return null;
 }
        throw error;
    }
}

// Задание 3: appendFile сохраняет предыдущие записи журнала.
async function addLog(message) {
    try {
        await fs.appendFile(path.join(__dirname, 'storage', 'activity.log'),
            `${new Date().toISOString()} ${message}\n`, 'utf8');
    } catch (error) { throw new Error(`Запись журнала: ${error.message}`); }
}

// Задание 4: оригинал удаляется только после успешной записи копии.
async function makeCopy() {
```

Листинг 1. index.js (часть 1)

index.js

```
    try {
        const source = path.join(__dirname, 'source.txt');
        const content = await fs.readFile(source, 'utf8');
        await fs.writeFile(path.join(__dirname, 'copy.txt'), content, 'utf8');
        await fs.unlink(source);
        return true;
    } catch (error) {
        if (error.code === 'ENOENT') { console.log('Исходный файл не найден'); return
false; }
        throw error;
    }
}

// Задание 5: access необходимо ожидать внутри try, иначе catch не сработает.
async function checkFiles(files = ['copy.txt', 'index.js', 'tasks.txt', 'missing.txt'])
{
    const result = await Promise.all(files.map(async file => {
        try {
            await fs.access(path.join(__dirname, file));
            return { file, exists: true };
        } catch (error) {
            if (error.code !== 'ENOENT') throw error;
            return { file, exists: false };
        }
    }));
    result.forEach(({ file, exists }) => console.log(`${file}: ${exists ? 'существует' :
 'не существует'}`));
    return result;
}

async function start() {
    await setup();
    await readUpper();
    await addLog('Программа запущена');
    await makeCopy();
```

Листинг 2. index.js (часть 2)

index.js

```
    await checkFiles();
}

module.exports = { setup, readUpper, addLog, makeCopy, checkFiles };
if (require.main === module) start().catch(error => { console.error(error.message);
process.exitCode = 1; });
```

Листинг 3. index.js (часть 3)

Проверка результата

Проверено: 05.11.2026 13:44 (Алматы). Node.js v26.12.1.

Приведены фактические результаты проверки. Полный журнал: verification.json.

• setup ожидает создание папки и файла

• Нет tasks.txt → понятное сообщение без падения

• Журнал содержит дату и сохраняет предыдущие записи

• Ошибка записи копии сохраняет оригинал; успешная запись предшествует удалению

• Promise.all различает существующий и отсутствующий файл

Результат

Проверены обязательные операции и указанные ошибочные сценарии; результаты сохранены вместе с исходным кодом.